



P2P Systems and Blockchains

-Lab-

Damiano Di Francesco Maesa
9/3/2026
damiano.difrancesco@unipi.it



UNIVERSITÀ
DI PISA

bitcoinj - transactions

Clear description of a transaction fields:

<https://dev.to/thunderbiscuit/deconstructing-a-bitcoin-transaction-4l2n>

Segwit recap

<https://learnmeabitcoin.com/technical/upgrades/segregated-witness/>

bitcoinj - transactions

```
1 public static void printTxInfo(Transaction tx) throws InterruptedException {
2     //txHash, isCoinbase, weight, hasWitness:
3     StringBuilder line = new StringBuilder();
4     boolean isCoinbase = false;
5     boolean hasWitness = false;
6     line.append(tx.getTxId().toString());
7     line.append(",");
8     if (tx.isCoinBase()) {
9         isCoinbase = true;
10        line.append("1");
11    } else {
12        isCoinbase = false;
13        line.append("0");
14    }
15    line.append(",");
16    line.append(" " + tx.getWeight());
17    line.append(",");
18    if (tx.hasWitnesses()) {
19        hasWitness = true;
20        line.append("1");
21    } else {
22        hasWitness = false;
23        line.append("0");
24    }
25    System.out.println("General info : " + line.toString());
26    line = new StringBuilder();
27    if (isCoinbase) {
28        //check if it has messages inside input scripts
29        boolean first = true;
30        for (TransactionInput ii: tx.getInputs()) {
31            if (first)
32                first = false;
33            else
34                line.append("\n");
35            line.append("Coinbase input script message? " + hexToAscii(bytesToHex
(ii.getScriptBytes())));
36        }
37    } else {
38        //not coinbase so there is at least one input
39        //prevTx_Id, prevTxPos, script:
40        boolean first = true;
```

```
41
42    for (TransactionInput ii: tx.getInputs()) {
43        if (first)
44            first = false;
45        else
46            line.append("\n");
47        line.append("Prev txHash " + ii.getOutpoint().hash().toString());
48        line.append("\nPrev txPos " + ii.getOutpoint().index());
49        line.append("\nscriptSig " + bytesToHex(ii.getScriptBytes()));
50        if (ii.hasWitness()) {
51            line.append("\nwitness " + ii.getWitness().toString());
52        }
53    }
54    System.out.println("Inputs :\n" + line.toString());
55    line = new StringBuilder();
56    //addr, amount, outScriptBytes
57    //there is always at least one output
58    boolean first = true;
59    for (TransactionOutput oo: tx.getOutputs()) {
60        if (first)
61            first = false;
62        else
63            line.append("\n");
64        byte[] outScript = oo.getScriptBytes();
65        String outAddr = ScriptParser.addrFromOut(outScript);
66        int outType = ScriptParser.typeFromOut(outScript);
67        if (outAddr == null) {
68            //writes '#UNKNOWN#' as address if not decodable
69            outAddr = "#UNKNOWN#";
70        }
71        line.append("Addr " + outAddr);
72        line.append("\nAmount " + oo.getValue().getValue());
73        line.append("\nscript " + bytesToHex(outScript));
74        line.append("\nscript " + printScriptAsOpCodes(outScript));
75        line.append("\nscript type " + ScriptTypeCustom.typeName(outType));
76    }
77    System.out.println("Outputs :\n" + line.toString());
78
79 }
```

bitcoinj - scripts

Scripts recap and opcodes list:

<https://learnmeabitcoin.com/technical/script/>

Detailed opcode lists:

<https://opcodeexplained.com/opcodes/>

<https://en.bitcoin.it/wiki/Script>

op_return:

<https://learnmeabitcoin.com/technical/script/return/>

bitcoinj - scripts

```
1 public static String printScriptAsOpCodes(byte[] script) {  
2     StringBuilder line = new StringBuilder();  
3     for (int i = 0; i < script.length; i++) {  
4         int val = Utilities.readUnsignedByte(script[i]);  
5         line.append(ScriptOpCodes.getOpCodeName(val));  
6         line.append(" ");  
7         i++;  
8         if ((val >= 1) && (val <= 75)) {  
9             for (int j = 0; j < val; j++) {  
10                 line.append(byteToHex(script[i+j]));  
11                 line.append(" ");  
12                 i++;  
13             }  
14         } else if (val == 106) {  
15             while (i < script.length) {  
16                 line.append(hexToAscii(byteToHex(script[i])));  
17                 line.append(" ");  
18                 i++;  
19             }  
20         }  
21     }  
22     return line.toString();  
23 }
```

bitcoinj - scripts

To get transaction raw hex

<https://blockchain.info/rawtx/a637ad18fabee7ad3ccd51e317091a6e16991311c0c9b83233b140b66b114448?format=hex>

To check if you decoded it correctly:

<https://www.blockchain.com/explorer/assets/btc/decode-transaction>

bitcoinj - scripts

```
1 public static void readRawTx() throws InterruptedException {
2
3     //String rawHex = "020000000180b98c54dbab5106d5a1449f4e5fdb9146deca1d48e93d66
4     6c5d9290b7c37a3f010000006b483045022100f0e32ceb205a5056694611afcffe4c1f0e63e9c5738
5     2607045ff2c3d9b5b7b3f0220111f0323e56d7462a9299833166569f1a68e1f5090b49bea64f541c4
6     94109c6c012102d0648f06a31d47112f1ff7848c85ce54b772c513bc3337c98f081c19d3dca260fff
7     fffff02006d7c4d000000001976a9147d463a046e3175142464740bad692fa0762a93e88accad5e5
8     f1b50000001976a914c98fc6bd9c2fd88533f28e6797cfa2a0a0e18ecf88ac00000000";
9
10    //first segwit
11    //dfcec48bb8491856c353306ab5febeb7e99e4d783eedf3de98f3ee0812b92bad
12    //String rawHex = "010000000001017405e391018c5e9dc79f324f9607c9c46d21b02f66da
13    baa870b4add871d6379f01000000171600148d7a0a3461e3891723e5fdf8129caa0075060cffffff
14    fff01fcf60200000000001600148d7a0a3461e3891723e5fdf8129caa0075060cfff02483045022100
15    88025cfdaf69d310c6fed11832edd9c19b6a912c132262701ad0e6133227d9202207d73bbf777abd
16    2aeae995d684e6bb1a048c5ac722e16de48bdd35643df7decf001210283409659355b6d1cc3c32dec
17    d5d561abaac86c37a353b52895a5e6c196d6f44800000000";
18
19    //opreturn
20    //d84f8cf06829c7202038731e544411adc63a6d4cbf8d4361b86698abad3a68a
21    String rawHex = "01000000178ded99d5bd03110a4e43d2cd7cddb3032af3e362d12ed0b8d
22    35cf811baf002556000000049483045022100fc323c40c6eb030c405bbacb478e6848b115c2bdbc5f7
23    8b275072ccecacaf8a02207102afeb0a16c2caf7dd07a06d642d1ee0286ebe5f7c4df092c78af4b0
24    4a44e101fffffffff02e8030000000000001976a9140910f9bbabaf5f653080ba23487d80a1dca614
25    388ac00000000000000000a6a084557204369616f2100000000";
26
27    byte[] ba = hexStringToByteArray(rawHex);
28    ByteBuffer bf = ByteBuffer.wrap(ba);
29    Transaction tx = Transaction.read(bf);
30    System.out.println("Tx ID: " + tx.getId());
31    System.out.println("Inputs: " + tx.getInputs().size());
32    System.out.println("Outputs: " + tx.getOutputs().size());
33    printTxInfo(tx);
34 }
```

Questions

